

Keywords: sparse attention • KV cache compression • end-to-end training • long-context LLM • differentiable gating

SAS: Simple Attention Sparsification via End-to-End Optimization

Hard Top-K Blocks Gradient Flow in Trainable Sparse Attention — Continuous Log-Space Gates Restore Differentiability and Outperform Baselines Under Tight KV Budgets

By Zhiwei Li, Lei Zhu, Hao Gu, Xiang Hu, Yan Wang, Haitao Mi, Sirui Han, Leo Liang, Zhijiang Guo (Tencent HY LLM Frontier; HKUST)

arXiv:2609.13141

Efficient long-context inference demands reducing attention from $O(L^2)$ to sub-linear cost by retaining only a sparse subset of KV entries per query. Trainable selectors promise to learn which tokens matter for downstream tasks — but a fundamental problem has gone unaddressed: the hard Top- K gate that implements sparsification is non-differentiable, severing gradient flow from the language modeling loss back to the selector. SAS (Simple Attention Sparsification) fixes this with continuous log-space gates injected directly into the attention softmax, enabling full end-to-end training with no architectural changes.

AT A GLANCE

Problem: Hard Top- K in trainable sparse attention blocks gradient flow; selectors train on surrogate losses misaligned with language modeling.

Why it matters: Long-context LLMs need 5–10% KV budgets to fit in memory; heuristic selectors lose 5–7 points on reasoning at these budgets.

Method: Inject gate $g_i = \sigma(s_i)$ as $\log g_i$ into attention logits before softmax; normalize against mandatory current block.

Result: Closes most of the gap to full attention at 10% KV budget on reasoning, long-context, and agentic benchmarks.

Evidence: LLaMA-3-8B and 70B, Mistral-7B; MATH-500, LongBench, WebArena, SWE-bench-lite.

Why Existing Approaches Fall Short

Sparse attention methods select a subset $\mathcal{S} \subset \{1, \dots, L\}$ of KV entries per query using a lightweight selector that outputs relevance scores $s_1, \dots, s_L$, then applies:

$$\mathcal{S} = \text{Top-}K(s_1, \dots, s_L)$$

The Top- K operation has zero gradient almost everywhere: $\partial \mathcal{S} / \partial s_i = 0$. Selectors are therefore trained with surrogate losses such as cosine similarity between selector output and the query — a proxy for what matters in generation. This mismatch is the root cause of quality degradation under tight budgets.

Core Mechanism: Log-Space Gate Injection

SAS replaces the hard gate with a continuous sigmoid gate:

$$g_i = \sigma(s_i) = \frac{1}{1 + e^{-s_i}} \in (0, 1)$$

The gate is injected into the attention logit before softmax:

$$\tilde{a}_{ij} = \text{softmax} \left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d}} + \log g_j \right)_j$$

Adding $\log g_j$ is equivalent to multiplying the pre-softmax probability by g_j :

$$\tilde{a}_{ij} \propto \exp \left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d}} \right) \cdot g_j$$

When $g_j \rightarrow 0$ a token is suppressed; when $g_j \rightarrow 1$ it is fully retained. Because softmax is differentiable, the gradient of the language modeling loss flows back through $\tilde{a}_{ij}$, through the gate g_j , and to the selector parameters:

$$\frac{\partial \mathcal{L}_{\text{LM}}}{\partial s_j} = \frac{\partial \mathcal{L}_{\text{LM}}}{\partial \tilde{a}_{ij}} \cdot \frac{\partial \tilde{a}_{ij}}{\partial g_j} \cdot \sigma'(s_j)$$

Technical Formulation

Gate-injected attention:

$$\tilde{\mathbf{a}}_i = \text{softmax} \left(\frac{\mathbf{q}_i K^\top}{\sqrt{d}} + \log \mathbf{g} \right), \quad \mathbf{g} = \sigma(\mathbf{s})$$

Selector: A 2-layer MLP or single attention head:

$$s_j = f_{\theta_s}(\mathbf{k}_j)$$

Training: Standard causal language modeling with gradients flowing through attention and the gate to θ_s :

$$\mathcal{L} = - \sum_t \log p_\theta(x_t \mid x_{<t})$$

Inference: Select top- K entries by s_j (hard), attend only to those. The selector has been trained with soft gradients.

Normalized Softmax Gates

The current block $\mathcal{C}$ (the most recent w tokens) is always retained in full. SAS normalizes historical gate scores against the attention mass consumed by $\mathcal{C}$:

$$\hat{g}_j = g_j \cdot (1 - A_{\mathcal{C}}), \quad A_{\mathcal{C}} = \sum_{j \in \mathcal{C}} \tilde{a}_{ij}, \quad j \notin \mathcal{C}$$

This calibrates the historical context budget to the remaining attention mass after the current block is satisfied, keeping sparsification levels consistent across sequence positions.

Learning or Inference Procedure

SAS is applied as a post-training fine-tuning step on a frozen or lightly-tuned backbone. The selector is initialized randomly or from heuristic scores and fine-tuned for as few as 500 steps on standard language modeling corpora. The MLP selector operates on cached keys and adds $O(L)$ overhead — negligible against $O(L^2)$ attention.

Experimental Results

Evaluated on LLaMA-3-8B, LLaMA-3-70B (long-context variants) and Mistral-7B-v0.3 at 10% KV budget:

Method	MATH-500	LongBench	WebArena
Full Attention	68.4	54.2	17.3
Quest	61.2	49.1	14.6
SnapKV	59.7	47.8	13.9
MagicPIG	63.1	50.4	15.1
SAS	66.8	53.1	16.9

SAS closes most of the gap to full attention at 10% KV budget, while prior trainable methods lose 5–7 points on MATH-500.

Ablations and Interpretation

Without log-gate (hard Top- K , end-to-end attempt): Training fails to converge; gate gradient is zero; performance equals Quest level.

Without normalized gate: Degradation of 3.2 points on LongBench; model over-attends to historical tokens and ignores the current block.

Selector depth: 2-layer MLP is sufficient; deeper selectors do not improve quality.

Budget sensitivity: SAS advantage is largest at tight budgets (5–10% KV); at 50% all methods approach full attention.

Reference

Zhiwei Li et al. **SAS: Simple Attention Sparsification via End-to-End Optimization of Context Ranking**. arXiv:2609.13141, September 2026. <https://arxiv.org/abs/2609.13141>